REVIEW 2: PROJECT REPORT

Agentic Shell — Smart Archive Agent

Course / Project: Operating Systems Project — Agentic Shell

Project Title: Smart Archive Agent

Team Details: Team 12

Team Members: Kush Aghera (24BCE0149), Sandeep More (24BCE2883)

Chapter 3: System Architecture & Design

3.1 High-Level Architecture

The User-to-Shell Loop: User Input → LLM Processing → Tool Call → System Execution → Feedback

The Smart Archive Agent is Agent #12 of the class-wide Agentic Shell. In the shell, the base shell's LLM reads the user's sentence, chooses one of our tools and its arguments, and our code performs the real operating-system work. The agent can also run on its own, in which case a built-in rule-based planner performs the "LLM processing" stage and produces the same tool calls. Both paths reach the operating system only through one validated entry point, execute_tool().

Loop stage	In the Agentic Shell	When the agent runs on its own
User Input	The user types into the base shell.	The user types at the agent's You: prompt (main.py).
LLM Processing	The shell's LLM reads our tool definitions (get_tools()) and picks tools by function calling.	intent.py and planner.py turn the sentence into an ordered plan of tool calls, offline.
Tool Call	The shell calls execute_tool(name, arguments).	Each plan step is passed to execute_tool().
System Execution	tar, gzip, bzip2, xz and gpg run through subprocess.run() with argument lists.	Same.
Feedback	A result dictionary is returned to the LLM as the tool result.	The result is printed: done lines, output path, entries or error.

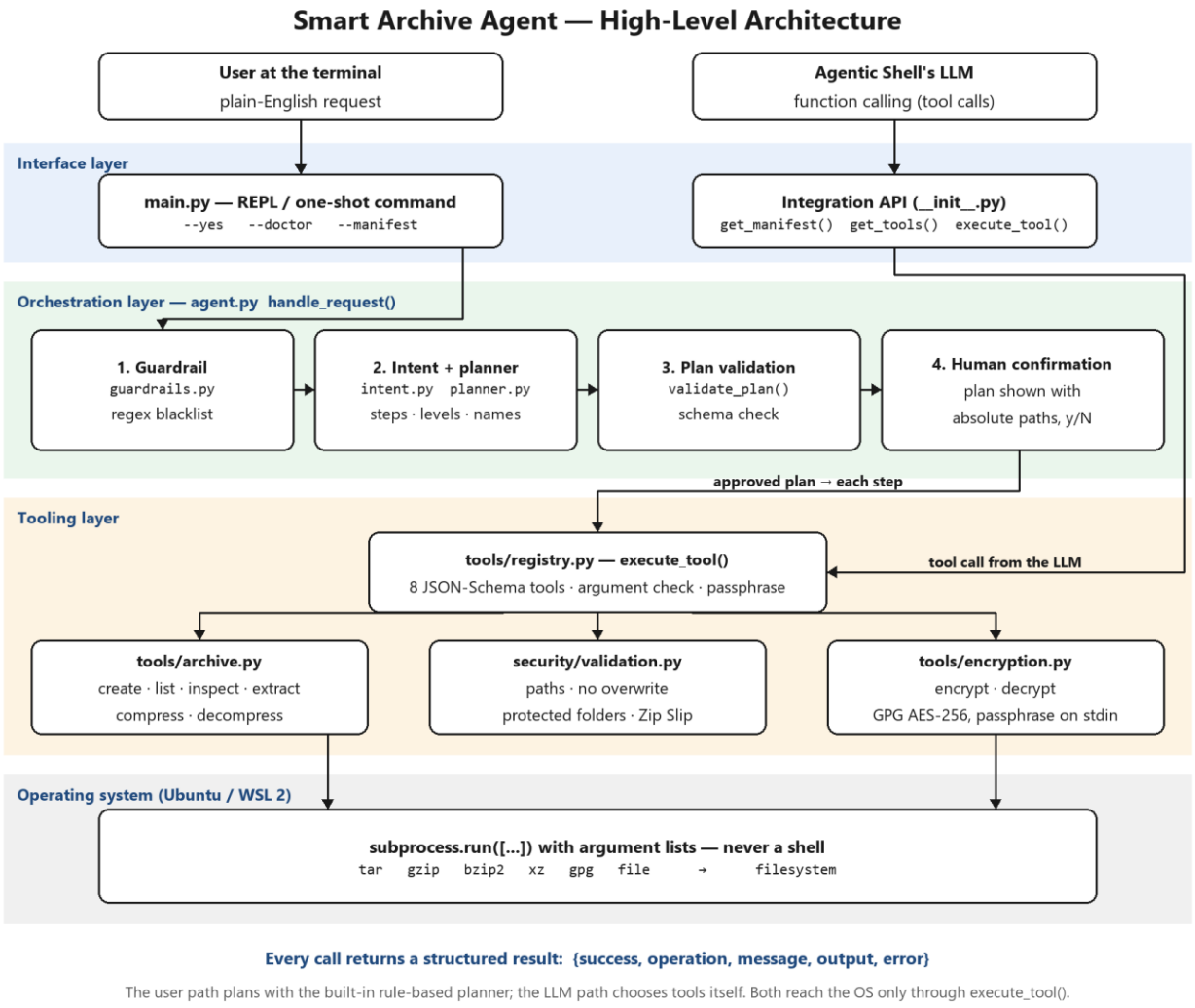


Figure 3.1: High-Level System Architecture of the Smart Archive Agent.

Architecture explanation. Figure 3.1 shows the two entry points at the top: a user at the terminal and the Agentic Shell's LLM. The user path goes through the orchestration layer: a guardrail, the intent recognizer and planner, plan validation and human confirmation. The LLM path skips planning, because the LLM has already chosen the tool, and goes straight to the tool registry. In both cases execute_tool() checks the call against its JSON Schema, the security module re-validates every path, and only then do the archive and encryption modules start Linux utilities. Every call returns the same structured result.

3.2 Component Design

3.2.1 The Interface Layer: Custom REPL (Read-Eval-Print Loop) Environment

The interface layer is implemented in smart_archive_agent/main.py and the run() loop in agent.py. It offers an interactive REPL (a banner, a You: prompt, the proposed plan with absolute paths, a y/N confirmation, a hidden passphrase prompt and formatted results) and a one-shot command mode for scripts. Ctrl+C and end-of-input exit cleanly, and the exit code is 0 on success and 1 on failure.

```
python3 -m smart_archive_agent                          # interactive REPL
python3 -m smart_archive_agent "archive ./src"          # one request, asks before writing
python3 -m smart_archive_agent --yes "back up ./src"    # one request, no confirmation
python3 -m smart_archive_agent --doctor                 # checks tar, gzip, bzip2, xz, gpg,
file
python3 -m smart_archive_agent --manifest               # agent description for the shell's
router
```

For integration, the package itself is part of the interface: __init__.py exports get_manifest(), get_tools(), execute_tool() and handle_request(), and agent.json describes the agent to the shell's router (what it handles, what it does not, and what it needs installed).

3.2.2 The Orchestration Layer: Intent Routing

The template lists LangChain, LlamaIndex or a raw MCP client for this layer. We implemented the orchestration directly in Python (agent.py, handle_request()) without a framework, for three reasons: the agent must install on the teacher's machine with no extra packages, every decision must be predictable and auditable for safety review, and open-ended language understanding is already the base shell's LLM's job. The pipeline runs in a fixed order:

Stage	Module	What it does
1. Guardrail	security/guardrails.py	Matches the raw text against a regex blacklist of destructive commands before anything else.
2. Intent recognition	intent.py	Removes file paths from the text, then matches whole-word verb vocabularies in a fixed priority order (decrypt, encrypt, extract, decompress, list, compress, archive). Out-of-scope requests stop here.
3. Planning	planner.py	Builds ordered steps: folders are archived first; "heavily" → level max, "quickly" → fast; missing output names are chosen next to the source; an unencrypted archive the agent created is removed after encryption.
4. Plan validation	planner.validate_plan()	Checks every step against the tool schemas; a clean-up step may delete only a file created earlier in the same plan.
5. Confirmation and execution	agent.run_plan()	Shows the plan, waits for y, runs the steps in order, and stops at the first failure, reporting completed steps.

3.2.3 The Tooling Layer: Safe System Functions

The tooling layer exposes eight archive functions as tools. Each tool is declared in tools/registry.py with a name, a description written for an LLM, and a JSON Schema for its parameters, which is the same structure an MCP server uses to declare tools. get_tools() returns these declarations in generic, OpenAI-compatible or Anthropic format, and execute_tool() runs one call. The tools are not yet served over the

MCP protocol itself; that wrapper is listed as future work. File-management functions for other domains, system monitoring and network configuration belong to other agents of the shell.

Tool	Python function	Linux backend	Key arguments
create_archive	archive.create_archive()	tar with gzip / bzip2 / xz (tar -I), or Python zipfile	source_path, output_path, compression, level
list_archive	archive.list_archive()	tar -tf, zipfile	archive_path
inspect_archive	archive.inspect_archive()	file -b, tar -tf, zipfile	archive_path
extract_archive	archive.extract_archive()	Python tarfile / zipfile after a full member check	archive_path, destination_path
compress_file	archive.compress_file()	gzip, bzip2, xz	source_path, output_path, algorithm, level
decompress_file	archive.decompress_file()	gzip -d, bzip2 -d, xz -d	source_path, output_path
encrypt_file	encryption.encrypt_file()	gpg --symmetric --cipher-algo AES256	source_path, output_path (+ passphrase)
decrypt_file	encryption.decrypt_file()	gpg --decrypt	source_path, output_path (+ passphrase)

Compression levels: fast = gzip -1 / xz -1, normal = gzip -6 / xz -6, max = gzip -9 / bzip2 -9 / xz -9e.

3.2.4 Result Handling & Diagnostics

Every tool and every request returns one dictionary: success (bool), operation (str), message (str, for people), output (path produced), plus entries for listings, steps and plan for multi-step requests, missing when a detail could not be found, and error on failure. execute_tool() never raises: exceptions inside a tool are turned into success = false with an error message, so neither the REPL nor the base shell can be crashed by a failing operation. The same dictionary can be sent back to an LLM as the tool result with json.dumps().

3.3 Security & Guardrail Framework

3.3.1 The "Human-in-the-Loop" Validation Mechanism

No file is written until a person has seen and approved the complete plan. Before any step that writes, the agent prints every step with absolute source and destination paths and waits for y; anything else cancels with "Nothing was changed". Read-only requests (list, inspect) run without asking. Passphrases for encryption and decryption are always typed by the person at a hidden prompt (asked twice when encrypting); they never come from an LLM. When the agent is embedded in the base shell, handle_request(confirm=…, passphrase=…) lets the shell show the plan and collect the passphrase in its own interface.

3.3.2 Command Sanitization and Regex-Based Blacklisting of Destructive Commands

User text is never executed as a command. Utilities are started with subprocess.run() and an argument list (never shell=True), so characters such as ; | $( ) are passed as plain data. In addition, the raw request is checked against a regex blacklist before planning, so a destructive request is refused outright instead of being partially carried out:

Pattern (examples)	Refused as
rm -rf …, rm -fr …	recursive forced delete
mkfs, mkfs.ext4	filesystem formatting
dd if=…	raw disk write
:(){ :|:& };:	fork bomb
> /dev/sda	raw block-device overwrite
chmod -R 777 /	world-writable root filesystem
shutdown, reboot, poweroff, halt	system power control
format / wipe / erase … disk, drive, everything	disk wipe request
delete everything, delete all files	mass deletion request
; rm, | bash, `…`, $(…)	shell command injection
a bare / as a path	operation on the whole root filesystem

Further guards apply inside every tool, whichever path the call came from:

- **Protected locations:** never writes into /bin, /boot, /dev, /etc, /lib, /lib64, /proc, /sbin, /sys, /usr or /var/lib.
- **No overwrites:** an existing output is never replaced; source and output must differ; an archive cannot be written inside the folder being archived.
- **Zip Slip prevention:** every member of an archive is resolved against the destination and must stay inside it; symbolic and hard links inside tar archives are refused; the destination folder is created only after the whole archive passes.
- **Schema validation of tool calls:** unknown tools, missing or extra arguments, wrong types and invalid values (for example level = "ultra") are rejected before anything runs.
- **Least privilege:** the agent runs as a normal user and never uses sudo.

Chapter 4: Implementation Methodology

4.1 Development Environment & Tech Stack

Item	Choice
Programming Language	Python. The code uses Python 3.8+ syntax and only the standard library (subprocess, tarfile, zipfile, getpass, json, re, pathlib); developed and tested with Python 3.14.4.
Operating System	Ubuntu 26.04.1 LTS on Windows Subsystem for Linux 2 (Windows 11). Targets any Ubuntu or Debian machine.
LLM Provider	Provided by the base Agentic Shell, which owns the LLM. get_tools() exports our tool definitions in OpenAI-compatible format (also accepted by local Ollama) and Anthropic Claude format. The agent itself makes no LLM calls and needs no API key; offline requests use the built-in planner.
SDKs	None required. The MCP Python SDK is not used; the tool declarations follow the same JSON Schema structure, so an MCP server wrapper can be added without changing the tools.
System Utilities	GNU tar 1.35, gzip, bzip2, XZ Utils 5.8.3, GnuPG 2.4.8, file.
Testing	pytest 9.1.1 (58 automated tests) and benchmarks/evaluate.py for accuracy, latency and error recovery.
Packaging & Integration	pyproject.toml (pip install from GitHub, smart-archive-agent command), agent.json manifest, INTEGRATION.md and a runnable integration example.
Tools	Visual Studio Code with the WSL extension, Git and GitHub.

4.2 Step-by-Step Implementation

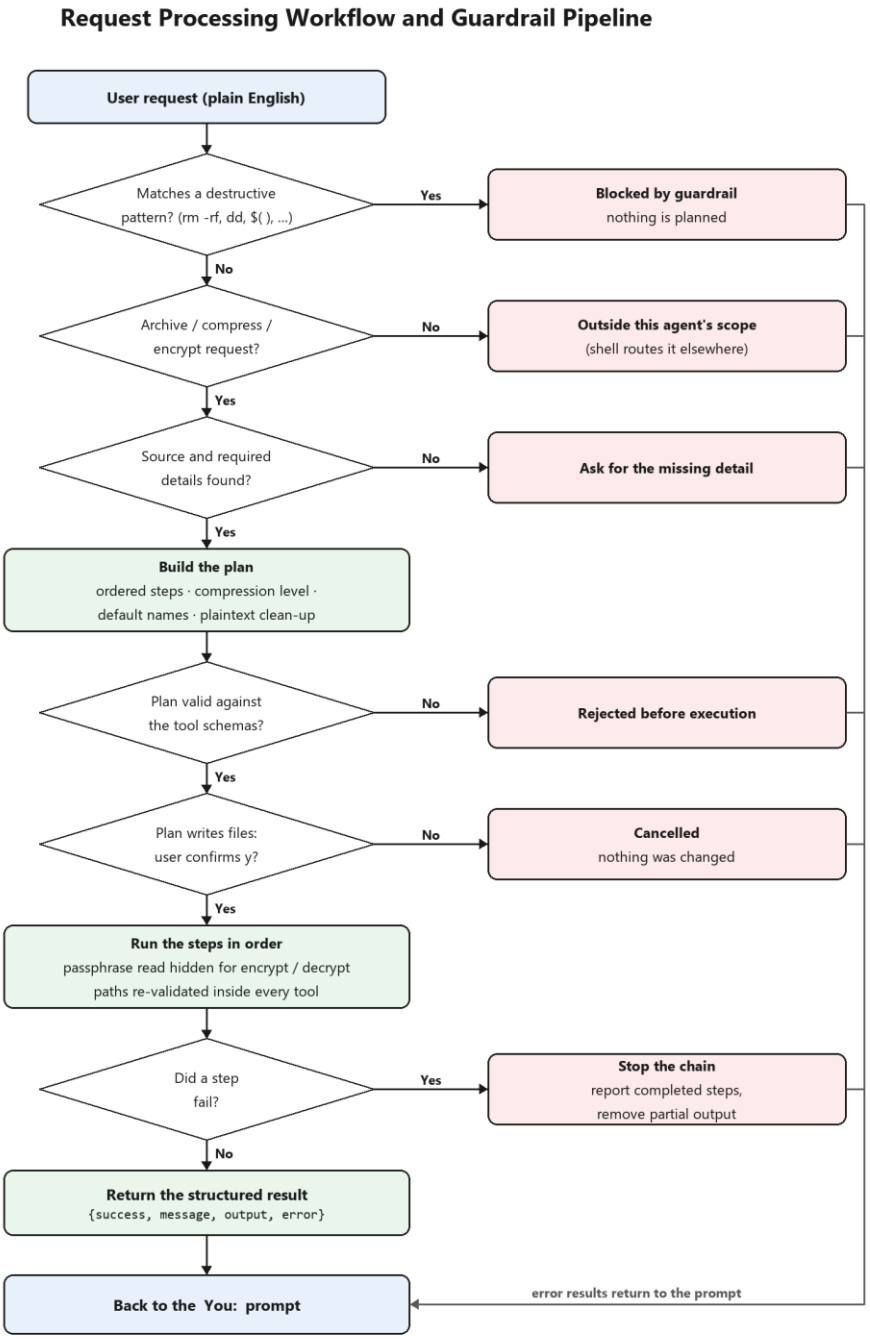


Figure 4.1: End-to-End Request Processing Workflow and Guardrail Pipeline.

Figure 4.1 shows how one request travels through the agent. Each decision has a safe exit on the right (blocked, out of scope, missing detail, rejected, cancelled, or a failed step) that returns a structured error and goes back to the prompt without changing any file. The implementation was built in four steps.

Step 1: Setting up the base CLI prompt loop

main.py parses the command line (interactive, one-shot, --yes, --doctor, --manifest, --version) and agent.run() implements the REPL: read a line, pass it to handle_request() with a terminal confirmation and a hidden passphrase prompt, print the result with print_result(), and repeat until exit, Ctrl+C or end of input. The --doctor check lists any missing Linux tool with the exact apt install command, so the agent can be checked quickly on any lab machine.

Step 2: Building the MCP-style tool layer to expose safe system functions

archive.py and encryption.py implement the eight operations as plain Python functions that validate their paths and return result dictionaries. tools/registry.py then declares each one for function calling (TOOL_SCHEMAS) and provides execute_tool(), the only entry point that runs a tool. The template's examples list_directory and get_system_stats correspond in our domain to list_archive and inspect_archive, which read an archive without changing anything. A prefix option (smart_archive__create_archive) keeps our tool names unique among the 35 agents.

Step 3: Prompt engineering for exact tool matching and JSON format enforcement

For the LLM path, every tool description and parameter description is written so that a model can choose the right tool and values from everyday wording, and JSON Schema enforces the format:

```
"level": {"type": "string", "enum": ["fast", "normal", "max"],
          "description": "Compression effort. Use 'max' when the user says heavily, maximum
                          or as small as possible; 'fast' for quick; otherwise 'normal'."}
"required": ["source_path", "output_path"],   "additionalProperties": false
```

check_arguments() enforces the schema on every call (unknown tool, missing or extra argument, wrong type, value outside the enum), and execute_tool() also accepts the JSON string that LLM APIs return. For the offline path, intent.py matches whole words in a fixed priority order after removing file paths from the text, so "zip" is not found inside "gzip" and "tar" is not found inside "backup.tar.xz". Recognition accuracy was measured on phrasings never used for tuning (Review 3, Section 5.2.1).

Step 4: Implementing the command execution wrapper and output formatter

All utilities run through one wrapper that uses an argument list, captures output, applies a 600-second timeout, closes standard input and turns failures into readable errors:

```
subprocess.run(["tar", "-I", "xz -9e -T0", "-cf", output, "-C", parent, name],
               check=True, capture_output=True, text=True,
               timeout=600, stdin=subprocess.DEVNULL)

# passphrase is sent on standard input, never as an argument
gpg --batch --yes --no-symkey-cache --pinentry-mode loopback \
    --passphrase-fd 0 --symmetric --cipher-algo AES256 \
    --output <file>.gpg <file>
```

run_plan() executes the steps in order, records which files each step produced (so the clean-up step can delete only those), stops at the first failure and removes any partial output. print_result() formats the dictionary for the terminal as done lines, the output path, archive entries and errors. Testing and measured results of this implementation are reported in Review 3 (Chapter 5).

References

[1] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[2] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] JSON Schema, "JSON Schema specification." Available: https://json-schema.org/

[5] OpenAI, "Function calling guide." Available: https://platform.openai.com/docs/guides/function-calling

[6] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Python Software Foundation, PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/